Secure Smart-Contract Permission & Key Management: Patterns, Formal Verification, and Java Tooling

Harshit Singla
Chitkara University Institute of Engineering & technology
Chitkara University
Punjab, India
harshit391.be22@chitkara.edu.in

Harshit Behal
Chitkara University Institute of Engineering & technology
Chitkara University
Punjab, India
harshit386.be22@chitkara.edu.in

Himanshu Shukla
Chitkara University Institute of Engineering & technology
Chitkara University
Punjab, India
himanshu408.be22@chitkara.edu.in

Dr. Anshu Singla
Chitkara University Institute of Engineering & technology
Chitkara University
Punjab, India
anshu.singla@chitkara.edu.in

***Abstract*—Managing keys poorly and leaving permission controls weak—that's still the main reason behind some of the worst smart-contract disasters. The Parity wallet freeze and the Ronin bridge hack together cost hundreds of millions of dollars, and in core, both came down to mishandled permissions. Surveys show there are plenty of smart-contract design patterns floating around, but hardly any tackle security head-on, and even fewer have been formally verified. We address that gap with three formally verified design patterns built for permission and key management: a k-of-n multi-signature wallet, a timelock controller for admin actions, Role Based Access Control (RBAC) module. Each pattern comes as a reusable Solidity template loaded with annotations, and we verify them through a multi-stage pipeline—static analysis with Slither, symbolic execution via Mythril, and formal proof with Certora / KEVM. Authors' tied all of these together with an open-source Java toolkit built on web3j, so contracts can be compiled, deployed, and continuously verified inside a standard CI workflow. The evaluation results shows that all ten core safety invariants—covering authorization, replay protection, timelock non-bypass, and privilege-escalation prevention—pass machine verification. Gas profiling on Ethereum testnets reveals per-operation overheads of 32–174% (roughly 1.3–2.7×) compared to single-admin baselines, which works out to $0.05 per transaction at typical gas prices—practical for mid- to high-value deployments. Adversarial simulations under partial key compromise back up the security claims, and early developer feedback points to nice audit.**

***Index Terms*—Smart Contracts, Design Patterns, key management, Access Control, Blockchain Security, Multisig, Solidity**

I. Introduction

Smart contracts on blockchains like Ethereum let people send money and control access without a middleman. But here's the problem: once you launch a contract, code is in stone. If you mess up even a little—especially with permissions—you're risking assets getting trapped / lost forever. Thought of Parity multi-signature disaster back in 2017. Someone triggered a self-destruct bug, and boom, around $150 M of Ether was locked up. Then there was the Ronin bridge hack in 2022 [1]. That one? Hackers got away with over $625 M because some folks didn't manage their validator keys carefully enough. Both times, it all boiled down to sloppy handling of keys and permisions.

When faced with a problem, people go for known design patterns. Azimi and colleagues [2] recently listed 144 patterns for smart contracts, but only 36 focus on security. Out of all those, just five deal with known vulnerabilities—that's barely 6 out of 94 types flagged by the OpenSCV taxonomy [3]. Even when you look at bigger lists like Six's 120 patterns [4], or collections on payment tokens from Lu [5], the same gap stands out: patterns for key and permission management are thin on the ground, and they haven't been checked thoroughly.

Researchers are clearly paying more attention to formal verification of smart contracts [6], but in practice, few actually use these methods. The Aptos team showed that it's possible to verify a big smart-contract framework with the Move Prover [7], and Antonino et al. [8] took this a step more by increasing their "Specification is Law" choice with model based on refinements [9]. These are both impressive milestones. Still, neither project offers reusable Solidity templates of permission patterns, and developers don't yet have a continuous-verification toolchain built for everyday use.

Researchers have dug into secret sharing [10], custodial strategies [11], off-chain access controls [12], and reactive ways to key-loss recovery [13]. Each tackles a different part of managing keys, but none delivers preventive, pattern-driven, or formally verified permission systems.

We bring together three different areas: pattern catalogues, formal verification, key management. And also By making them in union, we make a practical thing that makes smart-

contract design fast and good. Here's what the present study contributes:

1) Introduced three design patterns, all formally specified. These patterns cover smart-contract permission and key management: a k-of-n multisignature wallet, a timelock controller, and a role-based access control module. Each pattern addresses a distinct need with clear boundaries.
2) supplied reusable Solidity templates, but they aren't just bare code—they're packed with annotations. Alongside them, the authors' provide Certora Verification Language (CVL) specifications define ten core safety invariants. This makes it much easier to reason of security and good in real-world deployments.
3) Built a multi-stage verification pipeline. It pulls in tools like Slither, Mythril, and Certora, and ties them together using an open-source Java toolkit built on web3j [14].It automates the process thorough scrutiny at multiple levels.
4) Framework has been tested. Run formal proofs, measure gas costs, simulate adversarial things. This empirical evaluation grounds our work in both theory and practice.

Here's how paper tells. In Section II, you'll find an overview of related work. Section III lays out threat. The design patterns and the verification toolchain come next in Sections IV and V. Then, Section VI walks through our evaluation results. Section VII help with the problems. Finally, Section VIII wraps things up and points to where the research can go next.

II. Related Work

A. Smart-Contract Design Patterns

Azimi and colleagues [2] recently dove into smart-contract design patterns, cataloging 144 distinct patterns spread across Control, Maintainbility, Performence, and Security. Out of 36 security-related patterns, only five—like Checks-Effects-Interactions and Access Restriction, It focus specific vulnerabilities. These five only cover 6 out of the 94 security issues listed in the OpenSCV taxonomy [3]. That leaves developers with a huge gap: most security risks in smart contracts simply don't have pattern-based solutions yet.

Six et al. [4] compiled a taxonomy of 120 blockchain patterns, splitting them into on-chain and on/off-chain meta-categories, and then breaking those down further into 15 subcategories. For smart contracts its identified 11 patterns to security—things like Emergency Stop, Access Restriction, and Check-Effects-Interaction. But the team focused mostly on the conceptual side; they didn't offer formal verification / practical work you can plug this in code too.

Lu and colleagues [5] found 12 patterns for blockchain payment apps, mapping out how tokens move—from issuing them, transferring, to redeeming. They include multi-signature and escrow designs, but stay strict to payments. The collection doesn't tell on administretive tasks / key management problems.

Kannengießer and colleagues [15] took a close look at smart-contract development on Ethereum, EOSIO, and Hyperledger Fabric. They pulled together 20 software patterns and mapped out the main hurdles developers face. Zou and his team [16] tackled the same territory, finding plenty of problems but also pointing to real opportunities—especially the demand for improved tools and trustworthy libraries. These studies dig deep into developer usability issues, though they don't dive into security or permission patterns. That piece is still missing.

B. Formal Verification - Smart Contracts

Tolmach and colleagues [6] put together a thorough review of formal specification and verification for smart contracts. They cover everything—model checking, theorem proof, symbolic execution, runtime verification—nothing gets left out. What really good is their point about the disconnect between existing tools and what developers actually use. To bridge that gap, they push for verification pipelines integrate directly into the tools developers rely on.

Park et al. [7] used Move Prover to verify the Aptos blockchain framework. They approached the task from two directions: high-level security goals and detailed function specifications. This let them catch actual problems, like arithmetic overflows lurking in the `block_prologue` function. Their research proves that formal verify things can work even at the scale of an entire smart-contract framework. Still, it's focused on the Move language and Aptos ecosystem—not Solidity or Ethereum's permission system.

Antonino et al. [8] introduced the "Specification is Law" approach. In this framework, formal specifications become fixed, not the code itself. A trusted deployer checks that a contract actually matches its specification before it goes live or gets updated. They tested this system on ERC-20 and ERC-1155 token standards, and it worked well. We're taking this idea further: we use formal invariants as good guide for defining permission patterns. These invariants set the rules—nothing else takes precedence.

C. Key Management and Recovery

Wu and colleagues [10] introduced a scheme to share secret key or can be keys of digital assets which made around smart contracts. They use Shamir's Secret Sharing to split up wallet private keys. The protocol tackles three main scenarios: protecting wallet security, have authorities having seize illegal crypto-assets, and handling inheritance of digitel assats. Their way to key custody, but it doesn't cover ongoing permission management once the contract is running.

Kandi et al. [17] proposed a decentralized blockchain-based key management protocol for heterogeneous IoT devices, demonstrating that on-chain key distribution scale on a lot of device types. Them saying reinforces the need for pattern-based key management but targets IoT rather than smart-contract administrative permissions.

Takei and Shudo [11] pragmatically analyzed key management practices for cryptocurrency custodians, evaluating multi-party computation and threshold sign schemes. They noted some gap in powerful security for smart-contract wallets.

Blackshear and colleagues [13] came up with KELP (Key-Loss Protection), a protocol with three stages: Commit → Reveal → Claim/Challenge. If you lose your key by accident, KELP gives you a way to get your assets back. The catch? It's *reactive*; you use it after some work go wrong. What we're doing takes the opposite approach. We're focused on *preventing* permission failures up front by using verified patterns. Both strategies work together—their method catches you after the fall, ours helps you avoid it altogether.

D. Secure Off-Chain Data Access

Goint et al. [12] introduced a symmetric-key encryption protocol protect on data stored off-chain in blockchain consent. They keep encryption keys anchored on the blockchain, releasing them only to those who have received consent from the data owner. While their way deals with key management, it mainly focuses on controlling data access when consent processes, not on managing on-chain administrative permissions.

E. Summary

Table I contrasts the scope of related work with our contribution. Nobody else offers reusable Solidity templates for key and permission patterns, verifies it, and bundles all that inside a developer-friendly CI toolkit.

III. Threat Model and Problem Statement

Picture a smart-contract system run by n privileged key holders. The adversary's job? Get around the defenses and pull off administrative actions they aren't allowed to—like transferring treasury funds, upgrading contract code, or changing who has access.

Adversary capabilities. Adversaries can break security in some way:

- *Single-key compromise:* If they steal the private key of just one key holder—through phishing / malware, for example—they can compromise single-key schemes.
- *Minority-key compromise:* In multisignature setups like k-of-n, grabbing up to $k-1$ keys lets them get close but not fully control the process.
- *Logic exploitation:* Sometimes, attackers target bugs in the permission logic. Reentrancy, access-control bypasses, or missing authorization checks all fall into this category, and this problem can cause unwanted access or actions.
- *Governance attack:* Another approach is governance attack, where adversary tries to escalate roles without approval or bypass safety mechanism like timelocks.
- *Replay attack:* Replay attack is easy: the attacker resubmits a signed transaction that was already deemed valid, hoping the system will accept it again.

These tactics show that attackers don't keep with brute force—sometimes the right bug or gap is all they need.

Safety invariants. We designed our patterns to lock in some key protections, even when looking to adversary model we described.

P1 You need at least k unique, valid signatures to set off any privileged action.

P2 One address alone can't pull the trigger on privileged functions.

P3 Messages with signatures get accepted just once—no replays, so replay protection is baked in.

P4 Changes the owner sets up demand multisig approval; no shortcuts.

P5 Operations in the queue wait—nothing jumps ahead of its schedule.

P6 Only the roles we authorize can cancel a queued operation.

P7 Operations that have expired, even after their grace period, don't get to execute.

P8 Role R can only be granted, revoked by its role administrator.

P9 When a role transfers, the recipient has to explicitly accept it.

P10 There's no sneaky way to escalate privilege in the role graph—no loopholes.

Scope exclusions. There are some things we don't try to cover. If a person who is attacker manages to compromise a full k-of-n majority of keys—essentially, collusion among most key holders—our patterns can't stop that. We also no address bug in the EVM itself or in Solidity compiler, and physical attacks on key holders are outside our scope.

IV. Design Patterns

Pattern is presented with its *intent*, *structure*, and the *formal properties* it guarantees (referencing P1–P10 from Section III). Simplified Solidity excerpts illustrate key design decisions; the complete annotated templates are available in our repository.[1]

A. Multisignature Wallet (k-of-n)

Intent. Prevent a single compromised key from executing any critical operation.

Structure. The wallet stores a set of n owner addresses and a threshold k. Privileged transactions are proposed off-chain, signed individually by owners, and submitted on-chain with the collected signatures. The contract reconstructs each signer via `ecrecover`, verifies uniqueness and membership, and executes the call only when $\geq k$ valid signatures are provided.

A monotonically increasing nonce binds each set of signatures to exactly one transaction, preventing replay. Owner-set modifications (adding, removing owners, or changing the threshold) are themselves gated by multisig approval, governance cannot be silently altered.

Properties enforced: P1 (authorization), P2 (no unilateral execution), P3 (replay protection via nonce), P4 (governance changes gated).

[1]Repository URL will be provided upon acceptance / can be shared with reviewers.

TABLE I
SCOPE COMPARISON WITH RELATED WORK. ✓ = ADDRESSED, (✓) = PARTIALLY ADDRESSED, – = NOT ADDRESSED.

Criterion	[2] Azimi	[4] Six	[6] Tolmach	[7] Park	[8] Antonino	[13] Blackshear	**Ours**
Reusable design patterns	✓	✓	–	–	–	–	✓
Key/permission focus	–	(✓)	–	–	–	✓	✓
Formal verification	–	–	(✓)	✓	✓	–	✓
Solidity templates	–	–	–	–	(✓)	–	✓
Developer CI tooling	–	–	–	✓	–	–	✓
Gas evaluation	–	–	–	–	–	–	✓
Usability assessment	–	–	–	–	–	–	✓

```solidity
function execute(
  address to, uint256 value,
  bytes calldata data,
  bytes[] calldata sigs
) external {
  bytes32 h = keccak256(abi.encode(
    to, value, data, nonce, block.chainid,
    address(this)));
  address prev;
  uint256 count;
  for (uint i; i < sigs.length; i++) {
    address s = ECDSA.recover(h, sigs[i]);
    require(isOwner[s], "not owner");
    require(s > prev, "duplicate/unordered");
    prev = s;
    count++;
  }
  require(count >= threshold, "below k");
  nonce++;
  (bool ok, ) = to.call{value: value}(data);
  require(ok, "call failed");
}
```

Listing 1. Simplified multisig execution (gas-optimized: off-chain signing, on-chain verification).

```solidity
function schedule(
  bytes32 id, uint256 eta
) external onlyRole(PROPOSER_ROLE) {
  require(eta >= block.timestamp + minDelay);
  require(timestamps[id] == 0, "exists");
  timestamps[id] = eta;
}

function execute(
  bytes32 id, address to,
  uint256 value, bytes calldata data
) external onlyRole(EXECUTOR_ROLE) {
  require(block.timestamp >= timestamps[id],
        "too early");
  require(block.timestamp <=
        timestamps[id] + GRACE, "expired");
  timestamps[id] = 1; // mark done
  (bool ok,) = to.call{value: value}(data);
  require(ok);
}
```

Listing 2. Timelock schedule and run (simplified).

B. Timelock Controller

Intent. Enforce a mandatory, publicly observable delay on administrative actions, make community scrutiny enable and emergency cancellation.

Structure. The controller manages a mapping from operation hashes to scheduled timestamps. An authorized proposer calls `schedule(target, value, data, delay)`, which records $eta = \texttt{block.timestamp} + delay$. After the delay has elapsed, an executor calls `execute(...)`, and the controller verifies `block.timestamp` $\geq eta$. A *grace period* parameter tell that stale operations expire and cannot be executed indefinitely. Authorized cancellers may void a queued operation at any time but it will be before someone do execution.

Properties enforced: P5 (delay respected), P6 (cancellation restricted), P7 (expiry enforced).

C. Role-Based Access Control (RBAC)

Intent. Map addresses to fine-grained capabilities with explicit, auditable transitions that avoid some high powers.

```solidity
function proposeGrant(
  bytes32 role, address account
) external onlyRole(getRoleAdmin(role)) {
  pendingGrant[role][account] = true;
  emit RoleProposed(role, account);
}

function acceptRole(
  bytes32 role
) external {
  require(pendingGrant[role][msg.sender]);
  pendingGrant[role][msg.sender] = false;
  _grantRole(role, msg.sender);
}
```

Listing 3. Two-step role grant (simplified).

Structure. Roles are represented as `bytes32` identifiers. Each role is have a designated *admin role* that controls who may give or take it. A two-step transfer mechanism (propose → accept) prevents suspicious grants: the prospective holder must actively claim the role.

Properties enforced: P8 (admin-gated grants), P9 (explicit acceptance), P10 (no escalation path).

D. Pattern Composition: Governed Contracts

The three patterns are designed to compose. A `Governed` base contract wires them together: the multisig wallet acts as the sole proposer on the timelock; the timelock serves as the sole executor for RBAC administrative actions. This layered architecture tell every permission change passes through both the multisig threshold and the timelock delay.

V. Formal Verification and Toolchain

A. Multi-Stage Verification Pipeline

Our pipeline applies three complementary analysis techniques in sequence.

Stage 1: Static analysis (Slither). Slither performs intra- and inter-procedural data-flow analysis to detect common anti-patterns (reentrancy, unprotected `selfdestruct`, state-variable shadowing). It serves like rapid first pass to catch structural defects before deeper analysis.

Stage 2: Symbolic execution (Mythril). Mythril explores execution paths up to a configurable depth, searching for concrete inputs not follow safety rules. We use Mythril to confirm the absence of exploitable paths for the authorization and replay-protection properties.

Stage 3: Formal proofs (Certora / KEVM). For the strongest guarantees, we encode properties P1–P10 as rules in the Certora Verification Language (CVL). Certora's Prover translates these rules and the Solidity bytecode into SMT constraints and either proves proparty for *all* inputs or returns a counterexample. For selected EVM-level properties we additionally use KEVM for bytecode-level verification.

B. Java Orchestration Toolkit

We provide an open-source Java toolkit built on web3j [14] that automates the end-to-end workflow:

1) **Compile:** Invoke `solc` to produce ABI and bytecode artifacts.
2) **Generate:** Run web3j code to create type-safe Java wrappers.
3) **Deploy:** Deploy contracts to a configurable target (local Hardhat node, Sepolia, or other testnet).
4) **Verify:** Invoke Slither, Mythril, and Certora as subprocesses, capturing their JSON / text output.
5) **Report:** Aggregate in a structured verification report with pass/fail CI gates.

The toolkit is packaged as a Maven project and integrates with standard CI systems. This put down barrier for enterprise Java teams that already use web3j for Ethereum interaction.

VI. Evaluation

A. Security Verification Results

Table II summarizes the formal-verification outcomes for all ten safety properties.

Slither reported zero high-severity and zero medium-severity findings across all three templates. Mythril, configured with a transaction depth of 3, found no exploitable execution paths for any property within the explored state space.

TABLE II
Formal verification results for properties P1–P10.

Prop.	Pattern	Tool	Result	Time (s)
P1	MultiSig	Certora	Verified	42
P2	MultiSig	Certora	Verified	31
P3	MultiSig	Certora	Verified	27
P4	MultiSig	Certora	Verified	48
P5	Timelock	Certora	Verified	21
P6	Timelock	Certora	Verified	18
P7	Timelock	Certora	Verified	24
P8	RBAC	Certora	Verified	33
P9	RBAC	Certora	Verified	19
P10	RBAC	Certora	Verified	64
Total				**327**

Slither: 0 high/medium findings (all contracts, <5 s total).
Mythril (depth = 3): 0 exploitable paths (avg. 83 s per contract).

TABLE III
Gas consumption for main operations (Hardhat local node).

Operation	Baseline	Pattern	Overhead
Deploy (multisig 3/5)	249,489	1,170,948	369.5%
Execute tx (multisig 3/5)	27,301	74,781	173.9%
Schedule (timelock)	—	52,945	—
Execute (timelock)	27,301	35,926	31.6%
Grant role (RBAC propose)	26,921	53,661	99.3%
Accept role (RBAC)	—	46,624	—

Baseline: single-admin contract with `require(msg.sender == admin)`.
Measured on Hardhat local node (Solidity 0.8.20, optimizer 200 runs).
"—" = no baseline equivalent (oparation new to the pattern).

B. Adversarial Simulation

We constructed three attack scenarios using Hardhat [18] test scripts:

1) **Single-key compromise in 3-of-5 multisig.** The attacker submits a transaction with 1 correct sign plus two forged signatures. The contract correctly rejects execution (P1 violation attempt).
2) **Timelock bypass.** The attacker calls `execute` before the scheduled time. The contract reverts with "too early" (P5 enforced).
3) **Unauthorized role escalation.** An address without the admin role calls `proposeGrant`. The modifier reverts (P8 enforced).

All three attacks were successfully blocked, consistent with the formal proofs.

C. Gas Profiling

We deployed the templates on a local Hardhat [18] node (Ethereum Mainnet fork) and on the Sepolia testnet. Table III reports gas consumption for main operations.

D. Developer Usability

We conducted a preliminary usability assessment with 8 developers (graduate students and research assistants with 1–3 years of Solidity experience). Participants were asked to

TABLE IV
RETROSPECTIVE MAPPING OF HISTORICAL EXPLOITS TO OUR PATTERNS.

Incident	Root Cause	Pattern	Mitigation
Parity Wallet 2017	Single-key `initWallet` call	P1, P2	Multisig prevents unilateral ownership change
Ronin Bridge 2022 [1]	5-of-9 validator key compromis	P5, P6	Timelock delay enables detection & cancellation
DAO 2016	Reentrancy + no admin override	P8, P6	RBAC emergency role + timelock cancel

integrate the multisig template into a sample DeFi treasury contract using our Java toolkit. We collected Likert-scale ratings (1–5) on four dimensions:

- *Template clarity:* mean 4.1 ($\sigma = 0.6$). Participants found the annotation-enriched templates significantly ease in audit than plain Solidity.
- *Integration effort:* mean 3.8 ($\sigma = 0.8$). Most participants completed integration within a single session, though two noted difficulty configuring the Certora prover key.
- *Verification output readability:* mean 3.9 ($\sigma = 0.7$). The multi-stage pipeline output was generally clear, with Slither results rated highest.
- *Overall security confidence:* mean 4.3 ($\sigma = 0.5$). Participants reported nice power when using formally verified patterns than unverified code.

Representative qualitative feedback included: "*The pattern annotations made it much clearer which invariants the contract enforces*" and "*The three-stage verification output gives more confidence than any single tool's results.*"

E. Retrospective Analysis: Historical Exploits

For the practical relevance of our patterns, we retrospectively analyzed three major smart-contract incidents and determined whether our patterns would avoided the attack.

Parity Wallet. The attacker called an unprotected `initWallet` function, taking sole ownership of the library contract. Our multisig pattern (P1, P2) will be required k signatures for any ownership change, blocking single-key exploits.

Ronin Bridge. The attacker compromised 5 of 9 validator keys and not taking care of time. A timelock (P5) on validator-set changes would have introduced a public delay window, allowing the community to find and cancel (P6) the malicious transaction before execution.

The DAO. While the main thing was reentrancy, the inability to freeze or pause the contract amplified the damage. An RBAC-gated emergency role (P8) combined with timelock cancellation (P6) would have made rapid response.

VII. DISCUSSION

Limitations. Our patterns and toolchain are specific to Solidity and the EVM. While the design-pattern concepts are conceptually portable to other smart-contract languages (e.g., Move, Ink!), the templates and CVL specifications would need to be rewritten for target. The formal proofs are also bounded by powers of the underlying SMT solvers; highly complex contracts may encounter timeouts.

The developer usability assessment was conducted with a small sample (8 participants). A larger, controlled checking with a lot of experience levels would strengthen the external validity of the usability findings.

Coverage. Our patterns address a focused subset of smart-contract security: key and permision management. They do not cover all 94 vulnerability classes in OpenSCV [3]. Increasing to additional security domains (e.g., reentrancy-guard patterns, oracle-manipulation defenses) is a natural direction.

Tool trust. Formal verification shifts trust from the contract to the verification tool and the specifications. Bugs in Certora, the Solidity compiler, or errors in the CVL specifications could undermine the guarantees. We mitigate this by layering three independent tools and by publishing all specifications for community review.

Implications. The pattern-plus-proof approach can be extended beyond key management. Any security-critical smart-contract concern which can express a state invariant or pre-/post-condition is amenable to this methodology. CI integration makes verification a *continuous* activity rather than a one-time audit.

VIII. CONCLUSION AND FUTURE WORK

We presented an integrated framework for secure smart-contract permission and key management comprising three formally verified design patterns (multisig wallet, timelock controller, RBAC), a multi-stage verification pipeline, and a Java-based CI toolkit. Our evaluation demonstrates that all ten safety invariants are machine-verified (total Certora proof time: 327 s), gas overhead ranges from 32–174% across operations (under $0.05 per transaction), adversarial simulations confirm robustness under partial key compromise, and a retrospective analysis shows the patterns would have mitigated the Parity, Ronin, and DAO incidents. Preliminary developer feedback (mean confidence 4.3/5) is positive.

Future work includes: (i) increasing pattern suite to cross-chain key management and bridge governance; (ii) investigating zero-knowledge proofs for privacy-preserving permission verification; and (iii) making a auto-generation tool that derives CVL specifications from annotated Solidity templates, further lowering the formal-methods barrier for practitioners.

REFERENCES

[1] Ronin Network, "Community alert: Ronin validators compromised," https://roninblockchain.substack.com/p/community-alert-ronin-validators, 2022.

[2] S. Azimi, A. Golzari, N. Ivaki, and N. Laranjeiro, "A systematic review on smart contracts security design patterns," *Empirical Software Engineering*, vol. 30, no. 95, 2025.

[3] F. Vidal, N. Ivaki, and N. Laranjeiro, "OpenSCV: An open hierarchical taxonomy for smart contract vulnerabilities," *Empirical Software Engineering*, vol. 29, no. 4, p. 101, 2024.

[4] N. Six, N. Herbaut, and C. Salinesi, "Blockchain software patterns for the design of decentralized applications: A systematic literature review," *Blockchain: Research and Applications*, vol. 3, no. 2, p. 100061, 2022.

[5] Q. Lu, X. Xu, H. M. N. D. Bandara, S. Chen, and L. Zhu, "Patterns for blockchain-based payment applications," in *Proceedings of the 26th European Conference on Pattern Languages of Programs (EuroPLoP)*. ACM, 2021.

[6] P. Tolmach, Y. Li, S.-W. Lin, Y. Liu, and Z. Li, "A survey of smart contract formal specification and verification," *ACM Computing Surveys*, vol. 54, no. 7, pp. 148:1–148:38, 2022.

[7] J. Park, T. Zhang, W. Grieskamp, M. Xu, G. Di Giacomo, K. Chen, Y. Lu, and R. Chen, "Securing Aptos framework with formal verification," in *4th International Workshop on Formal Methods for Blockchains (FMBC)*, ser. OASIcs, vol. 118, 2024, pp. 9:1–9:16.

[8] P. Antonino, J. Ferreira, A. Sampaio, and A. W. Roscoe, "Specification is law: Safe creation and upgrade of Ethereum smart contracts," in *Proceedings of the International Symposium on Leveraging Applications of Formal Methods (ISoLA)*, 2022.

[9] ——, "A refinement-based model for safe smart-contract evolution," *Science of Computer Programming*, vol. 237, p. 103148, 2024.

[10] C.-C. Wu, C.-T. Chang, I.-C. Lin, and M.-S. Hwang, "Research on blockchain secret key sharing and its digital asset applications," *International Journal of Network Security*, vol. 26, no. 1, pp. 160–166, 2024.

[11] Y. Takei and K. Shudo, "Pragmatic analysis of key management for cryptocurrency custodians," in *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2024.

[12] M. Goint, C. Bertelle, and C. Duvallet, "Secure access control to data in off-chain storage in blockchain-based consent systems," *Mathematics*, vol. 11, no. 7, p. 1592, 2023.

[13] S. Blackshear, K. Chalkias, P. Chatzigiannis, R. Faizullabhoy, I. Khaburzaniya, E. Kokoris Kogias, J. Lind, D. Wong, and T. Zakian, "Reactive key-loss protection in blockchains," in *Cryptology ePrint Archive*, no. 2021/289, 2021. [Online]. Available: https://eprint.iacr.org/2021/289

[14] web3j, "web3j – lightweight Java and Android library for Ethereum," https://github.com/web3j/web3j, 2024.

[15] N. Kannengießer, S. Lins, T. Töllich, and A. Sunyaev, "Challenges and common solutions in smart contract development," *IEEE Transactions on Software Engineering*, vol. 48, no. 11, pp. 4291–4318, 2022.

[16] W. Zou, D. Lo, P. S. Kochhar, X.-B. D. Le, X. Xia, Y. Feng, Z. Chen, and B. Xu, "Smart contract development: Challenges and opportunities," *IEEE Transactions on Software Engineering*, vol. 47, no. 10, pp. 2084–2106, 2021.

[17] M. A. Kandi, H. Lakhdari, D. E. Boubiche, and M. Kirik, "A decentralized blockchain-based key management protocol for heterogeneous and dynamic IoT devices," *Computer Communications*, vol. 191, pp. 447–459, 2022.

[18] NomicFoundation, "Hardhat – Ethereum development environment," https://hardhat.org, 2024.